

Selecting a Data Structure

CSC506 | DataStruct Lab | Assignment 1

Data structures organize values and define how software accesses and changes them. Their efficiency depends on the operations an application performs most often. DataStruct Lab compares a Python-list-backed stack, a deque-backed queue, and a custom singly linked list. Each stores n values in $O(n)$ space, but ordering, allocation, and traversal create different trade-offs. The bounds below assume constant-time value comparisons and fixed-size references. Big- O describes growth with input size, rather than an exact duration or a universal speed ranking.

A **stack** follows last in, first out (LIFO): the newest value is removed first. This supports undo histories, backtracking, and nested function-call behavior. In this implementation, pushing or popping at the Python list's end costs $O(1)$ amortized across a sequence; a resizing operation can individually cost $O(n)$. Peeking and reading the maintained length take $O(1)$. Searching may inspect all n values, so its worst-case cost is $O(n)$, although a top match returns immediately. A stack fits workloads centered on the most recent item, with occasional searches.

A **queue** follows first in, first out (FIFO): the oldest waiting value is served first. Scheduling, printer jobs, and message processing are natural examples. The lab wraps `collections.deque`, whose endpoint operations avoid shifting every remaining value after a removal. Enqueue, dequeue, peek, and size are $O(1)$, while a missing-value search is $O(n)$. This makes a queue appropriate when arrival order matters. This teaching wrapper does not implement concurrent producers, consumers, persistence, or scheduling priorities; those require additional design.

A **singly linked list** stores values in nodes connected by next references, with a head reference identifying the first node. Nodes can be allocated as needed without growing one contiguous array of references. Head insertion is $O(1)$, but locating an arbitrary value requires up to $O(n)$ traversal. The lab's delete-by-value operation removes the first matching node: relinking is $O(1)$ once its position is known, but finding that position makes deletion $O(n)$ in the worst case. Traversal takes $O(n)$ time and creates an $O(n)$ result list. Node objects and links add memory overhead; this design also lacks direct indexed access.

Selection should reflect operation frequency, access order, input size, memory cost, and response-time requirements. Favor a stack for recent-first access, a queue for arrival-order processing, and a linked list when head insertion and explicit node relationships suit the workload. Frequent lookup or indexing may favor a different structure. The lab's repeated missing-search measurements grow strongly with n , while short insertion timings fluctuate with overhead and allocation. These observations support investigating workload behavior; they do not prove asymptotic bounds or establish amortized cost from isolated pushes. No single structure is optimal for every application: select by required operations, then measure the actual implementation under representative workloads.